

**ENSE706 - Data Process and
Software Modelling**

Project Final Report

**Thanh Tran
24247169**

7/5/2026

Table of Contents

1. Introduction	1
2. Refined static and dynamic modelling	2
2.1. Refined UML Class Diagram	2
2.2. Sequence diagram	3
2.3. Activity diagram	6
3. Design Pattern Application	9
3.1. Factory Method pattern	9
i. Class diagram showing Factory Method pattern structure	9
ii. Corresponding C# implementation	10
3.2. Singleton pattern	13
i. Class diagram showing Singleton pattern structure	13
ii. Corresponding C# implementation	14
3.3. Adapter pattern	15
i. Class diagram showing Adapter pattern structure	15
ii. Corresponding C# implementation	17
4. Comparative design analysis	21
4.1. Changes in class structure and responsibility allocation	21
4.2. Improvements in cohesion and reduction in coupling	21
4.3. Impact on extensibility and maintainability	22
4.4. Consequences for C# implementation complexity	23
4.5. Accommodation of new requirements	24
5. Critical reflection	26
5.1. Design trade-offs introduced by design patterns	26
5.2. Limitations of the final design and implementation	27
5.3. Lessons learned from aligning UML models with C# code	28
6. Conclusion	30

1. Introduction

This report presents phase 2 of the Course Enrolment System for Auckland University of Technology (AUT), which extends and refines the system developed in phase 1. The project focuses on improving the system design through advanced UML modelling, design pattern application, and object-oriented refinement.

In phase 1, the system was analysed and designed based on the problems identified in AUT's existing system. Organisational and system analysis were conducted to identify stakeholders, departments, business processes, system scope, boundaries, and assumptions. Requirements engineering techniques such as problem statement analysis, document analysis, workflow observation, and stakeholder interviews were then used to define the functional and non-functional requirements of the system.

After the requirements analysis, UML requirements modelling was performed using use case diagrams, use case descriptions, and actor–use case relationships to represent system functionality and stakeholder interactions. A class diagram was then developed to model the system structure, including users, enrolment requests, courses, timetables, services, and external system interfaces. The design was also mapped into C# classes to support object-oriented implementation.

Phase 1 also included a partial C# implementation of core classes such as User, Student, Staff, RegistryStaff, Course, and EnrolmentRequest to demonstrate enrolment functions including course selection, enrolment submission, approval processing, and enrolment monitoring. Several design weaknesses were also identified, including dependency on external systems, risks of tight coupling, and possible performance issues during peak enrolment periods.

Building on this foundation, phase 2 focuses on refining the system design to improve abstraction, cohesion, modularity, maintainability, and extensibility using additional UML models and GoF design patterns. The refined design is then implemented in C#. The project also continues to apply version control practices using GitHub throughout the development process. GitHub link: <https://github.com/Thanh526/Thanh-Tran---24247169---Project-Report-and-Presentation>

2. Refined static and dynamic modelling

2.1. Refined UML Class Diagram

The class diagram shown below represents the initial Phase 1 design of the Course Enrolment System. Although it provides a reasonable overall structure of the system and identifies key entities, it is partially implementation-driven rather than conceptually clean. In addition, there are issues related to weak encapsulation in some classes, as well as unclear multiplicities and associations between certain domain objects. These limitations reduce the overall clarity, maintainability, and scalability of the design.

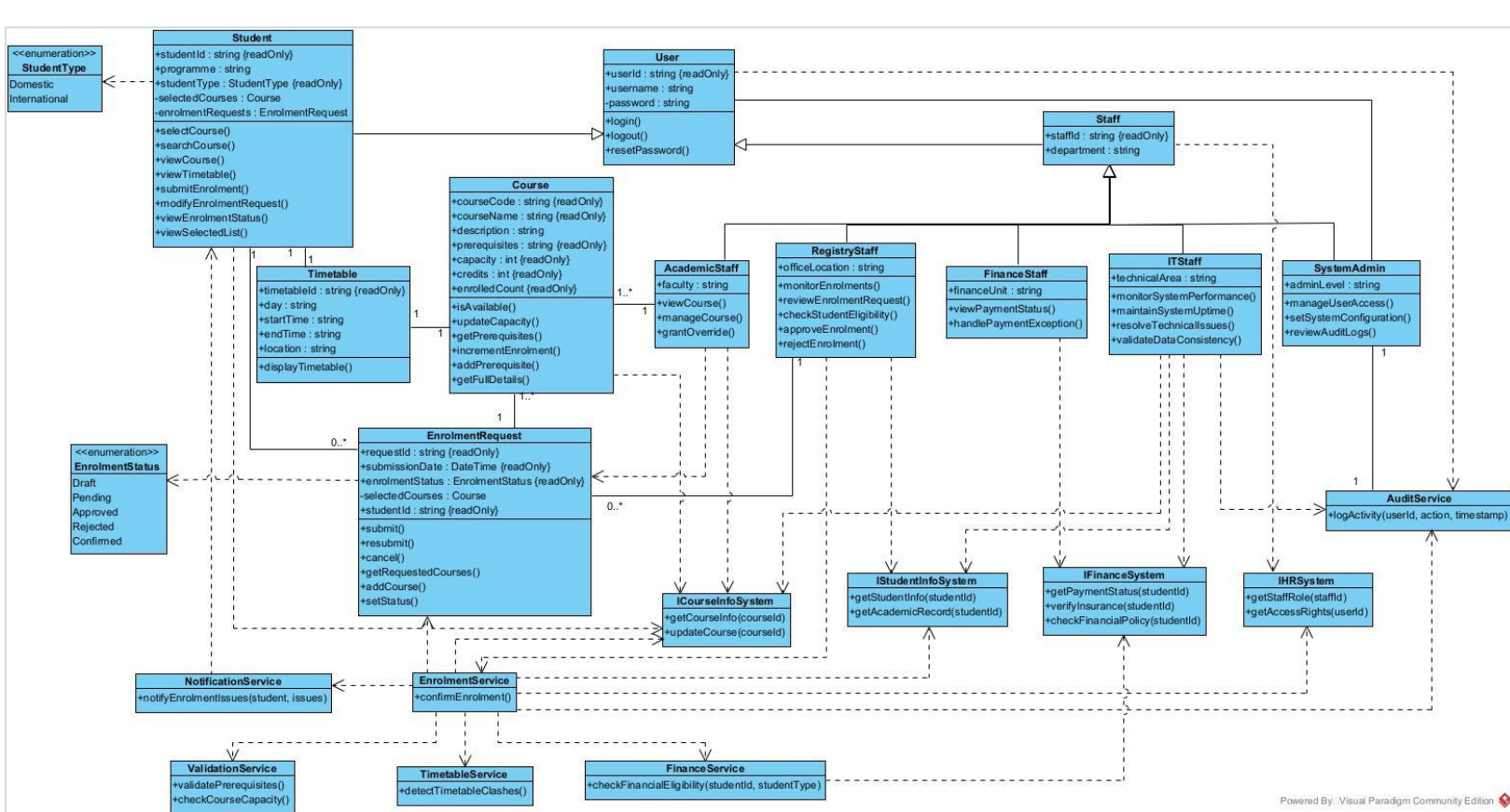


Figure 1: Phase 1's class diagram of Course Enrolment System

To address these issues, the class diagram has been refined by improving encapsulation and reducing implementation-level detail. Key attributes such as course capacity and identifiers are now private to protect data integrity. Some implementation-specific attributes and operations have been removed or simplified

registry staff. These processes were chosen because they represent the main workflow of the Course Enrolment System and involve interactions between multiple users, system classes, services, and external systems.

The first sequence diagram represents the student login process. It demonstrates how the student submits a login request through the system interface, which then communicates with the User class to authenticate the provided credentials. A loop structure is used to allow repeated login attempts until valid credentials are entered. Depending on the authentication result, the system records the login activity through AuditService and either displays the student dashboard for successful login or an error message for invalid credentials. After successful authentication, the student is able to continue accessing system functions such as searching, viewing, and selecting available courses.

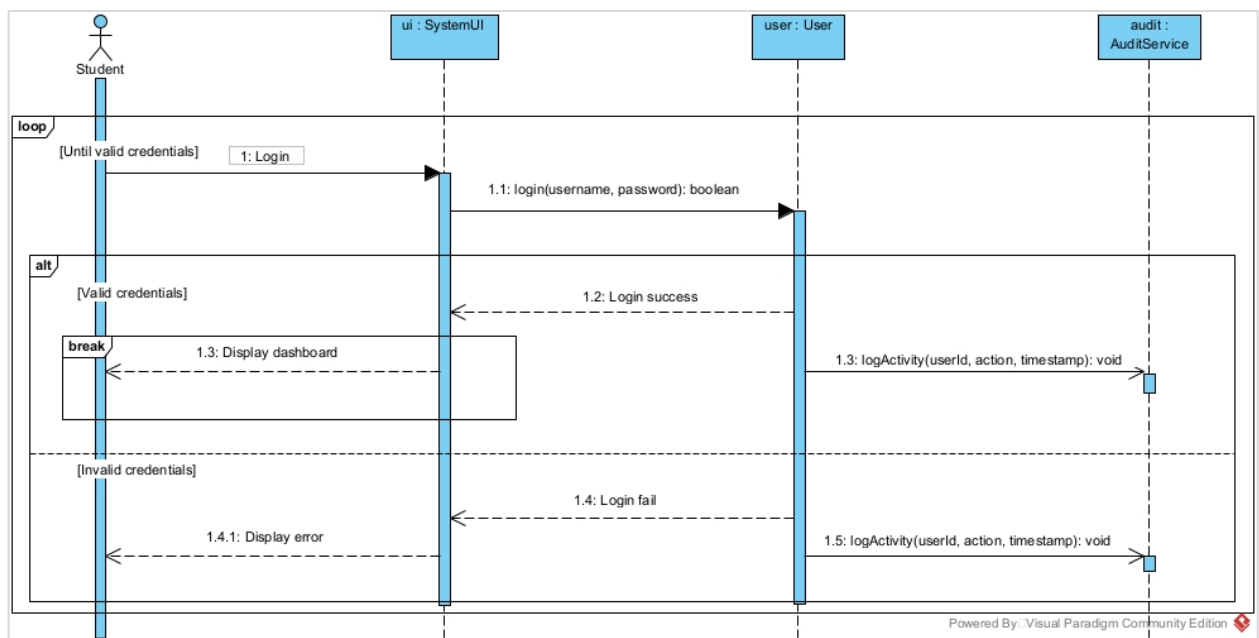


Figure 3: Sequence diagram of Student Login Process

The second sequence diagram represents the course selection and enrolment submission process. It illustrates how a student searches for available courses, views course details, and selects a course based on availability. The process is modelled as an iterative loop, allowing the student to repeat searching and selection until they decide to submit an enrolment request. When a course is available, an

EnrolmentRequest is created if it does not already exist, and the selected course is added to the request. Finally, the student submits the enrolment, where the request is finalised and processed by the system, and the action is recorded by the AuditService to ensure traceability and accountability within the system.

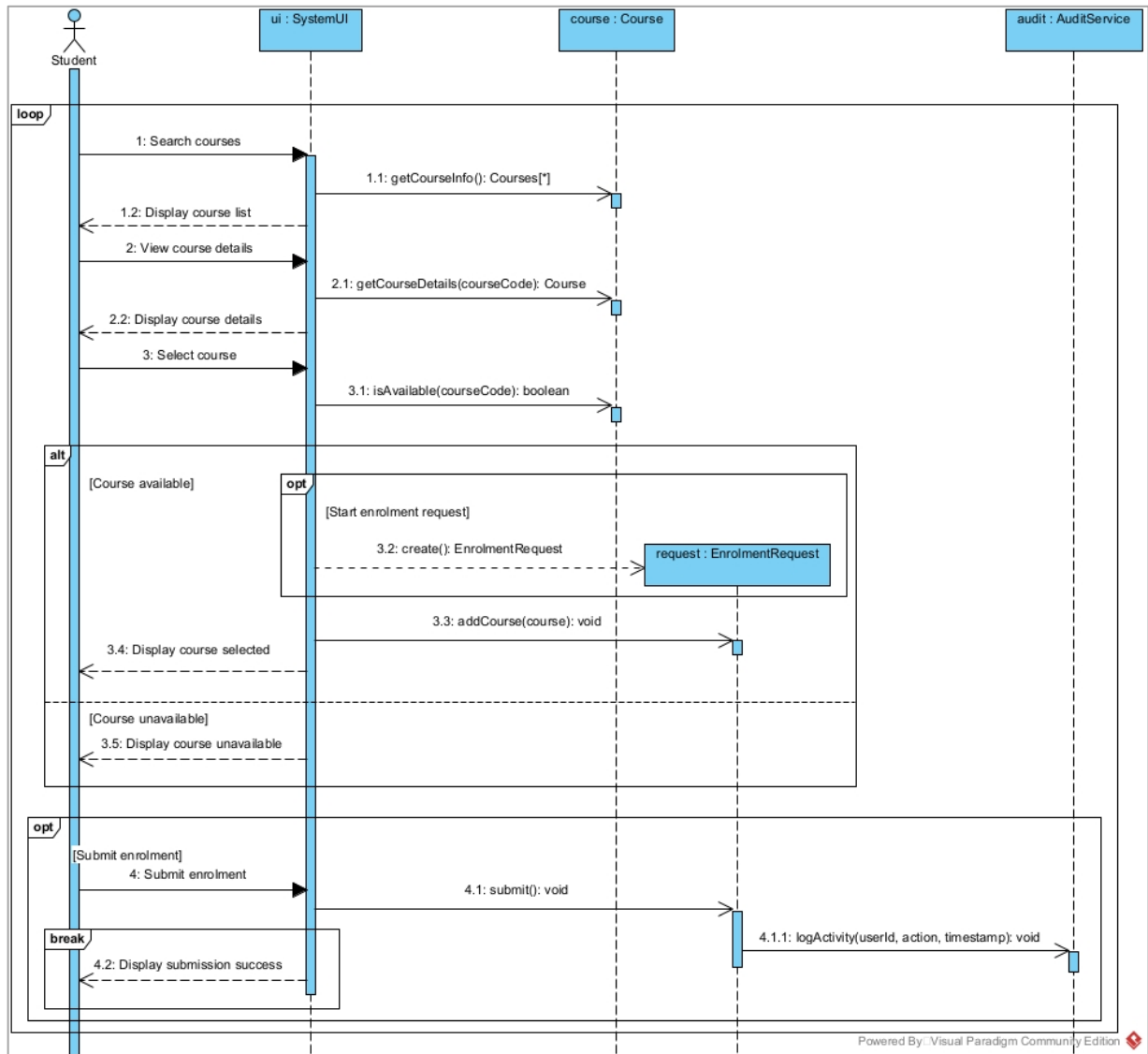


Figure 4: Sequence diagram of Course Selection and Enrolment Submission Process

The third sequence diagram represents the enrolment review and approval process performed by RegistryStaff. It shows how RegistryStaff reviews an enrolment request through the system UI, where EnrolmentService retrieves the request details along with the student's academic record, payment status, and insurance

information from external systems such as `IStudentInfoSystem` and `IFinanceSystem`. Based on the displayed information, `RegistryStaff` approves or rejects the request. `EnrolmentService` then updates the request status accordingly and records the action through `AuditService` to ensure traceability and accountability.

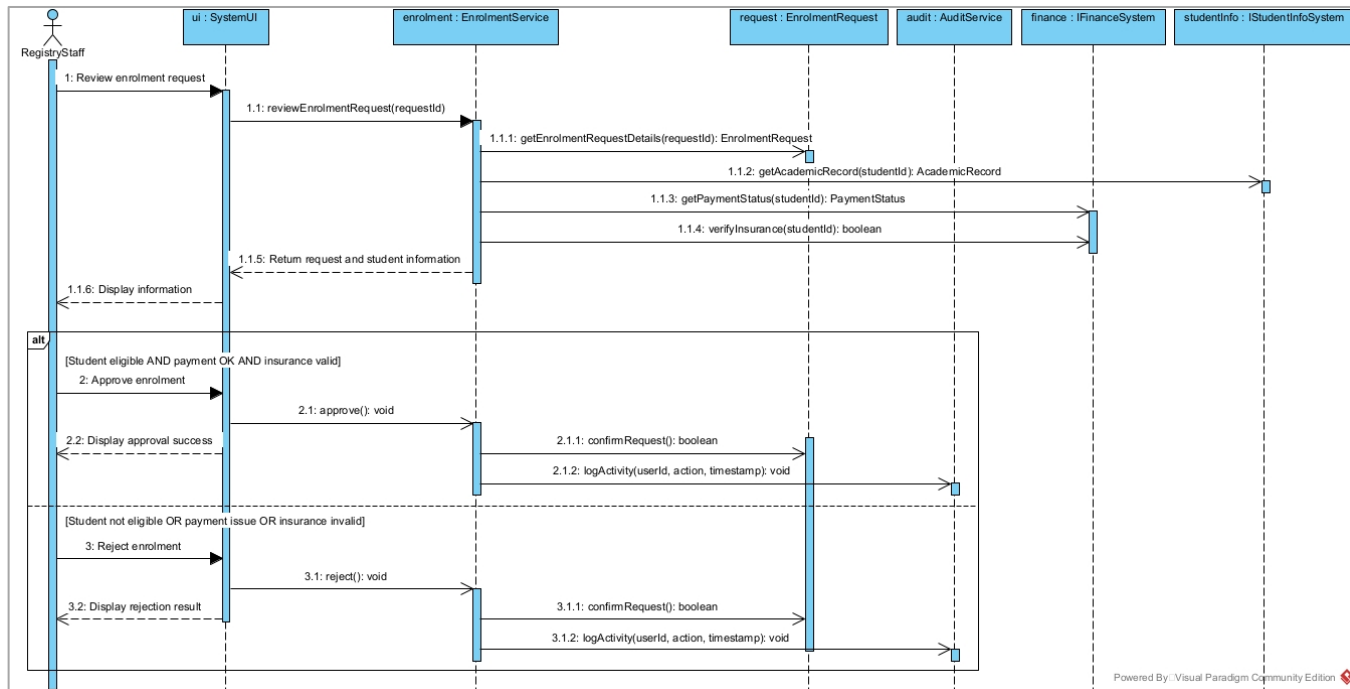


Figure 5: Sequence diagram of Enrolment Review and Approval Process

These sequence diagrams provide a clearer understanding of the system workflow by showing message ordering, interaction flow, and responsibilities between classes and external enrolment systems. They also improve consistency between the UML models and the C# implementation of the Course Enrolment System.

2.3. Activity diagram

To further represent the dynamic behaviour of the Course Enrolment System, activity diagrams were developed to model key system workflows. Unlike sequence diagrams, which focus on object interactions, activity diagrams provide a clearer representation of process flow, decision-making, and responsibilities between users, system components, and external services through the use of swimlanes.

The first activity diagram represents the student course enrolment process. It models the workflow from student login, course searching, viewing course details, selecting available courses, and submitting an enrolment request. Decision nodes are used to represent business rules such as login validation, course availability, and whether additional courses are added before submission. Once the enrolment is submitted, the request status is updated to pending and the activity is recorded through the AuditService.

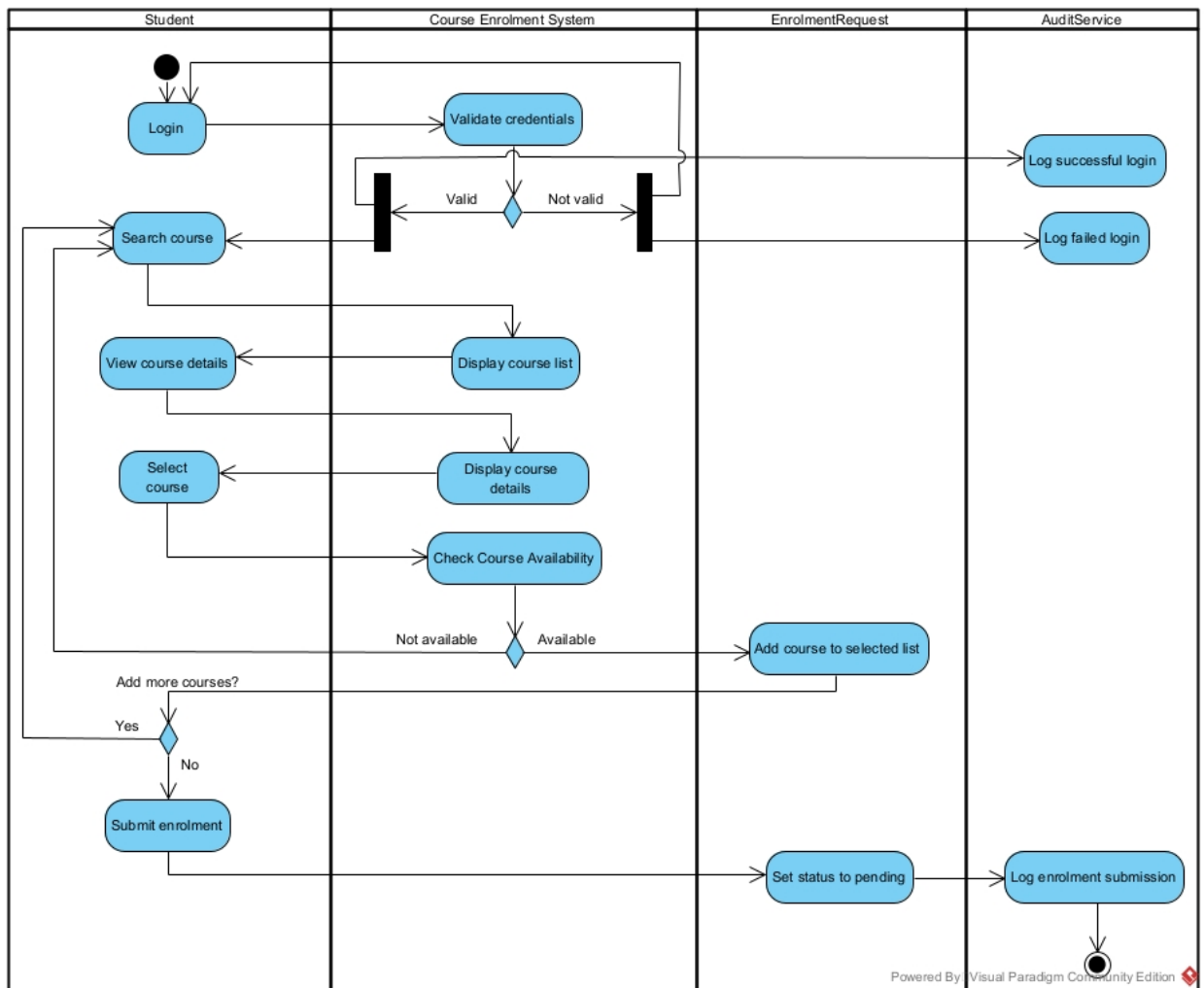


Figure 6: Activity diagram of Student Course Enrolment Process

The second activity diagram represents the Registry Staff enrolment review and approval process. It illustrates how RegistryStaff review enrolment requests and perform eligibility checks through IStudentInfoSystem and IFinanceSystem. Decision points are used to verify academic eligibility, payment status, and

insurance requirements. Based on the outcome, the enrolment request is either approved or rejected, and the action is recorded through the AuditService.

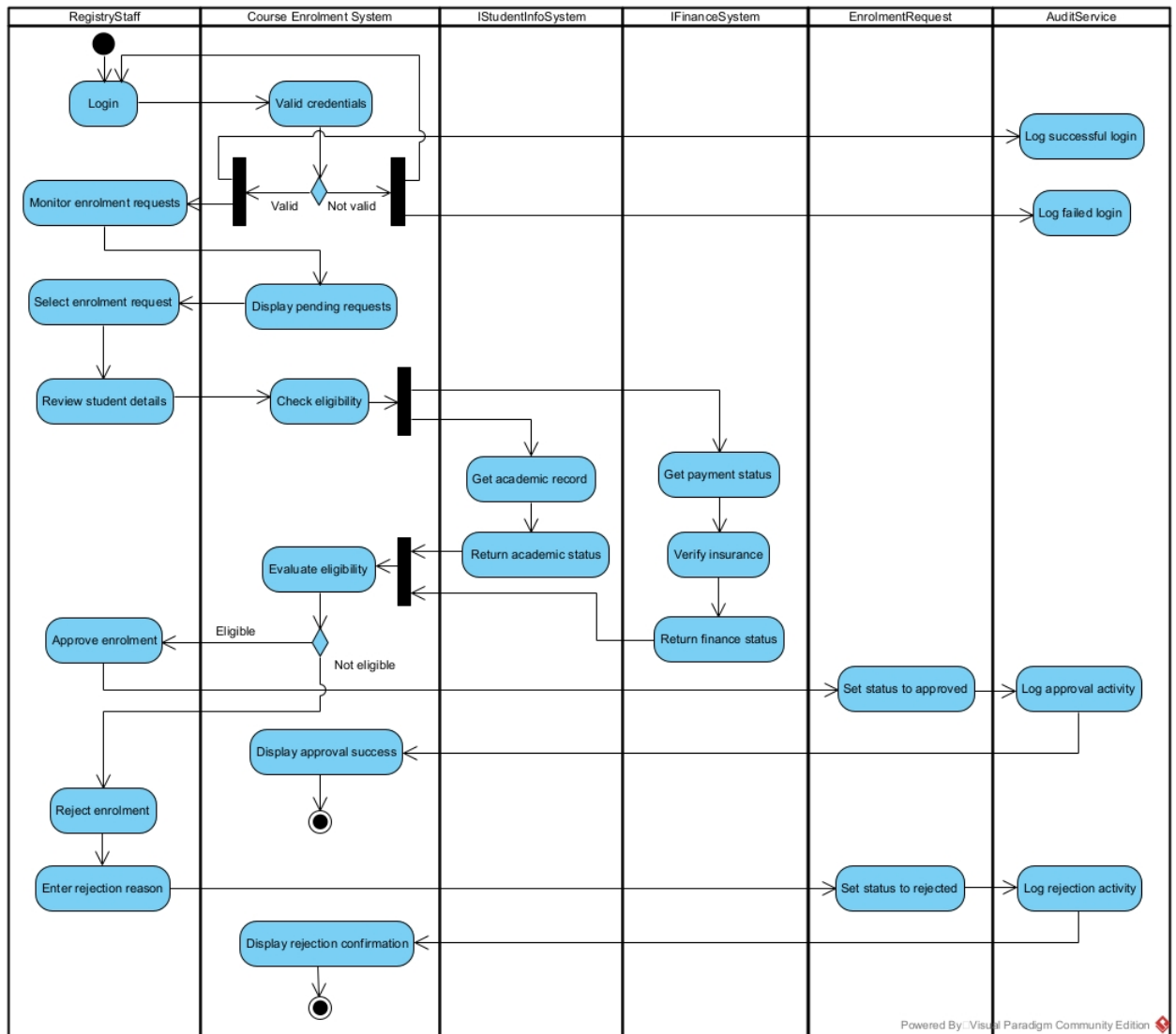


Figure 7: Activity diagram of Registry Staff Review and Approval Process

Overall, the activity diagrams provide a clearer understanding of the operational workflow of the Course Enrolment System and complement the sequence diagrams by focusing on business process flow and system behaviour from a process perspective.

3. Design Pattern Application

In this system, three GoF design patterns have been applied to improve modularity, maintainability, and overall system design quality. The selected patterns are Factory Method, Singleton, and Adapter. These patterns are used to address different design concerns such as object creation, global access control, and system integration with external services.

3.1. Factory Method pattern

The system requires the creation of different types of users and system actors, such as Student and various Staff types including RegistryStaff, FinanceStaff, ITStaff, and SystemAdmin. Directly instantiating these objects using new would tightly couple the client code to concrete classes and make the system difficult to extend when new user types are introduced.

The Factory Method pattern is applied to address this issue by delegating object creation to a factory class. This ensures that the system remains flexible, and extensible, as new user types can be added without modifying existing client code.

i. Class diagram showing Factory Method pattern structure

The figure below shows the changes in the class diagram after applying the Factory Method pattern. UserCreator acts as the creator class dependent on the User product, while concrete creator classes override the factory method to return different types of users. This improves system design quality by reducing tight coupling between client code and concrete classes, promoting adherence to the Open/Closed Principle, and making the system easier to extend and maintain without modifying existing code.

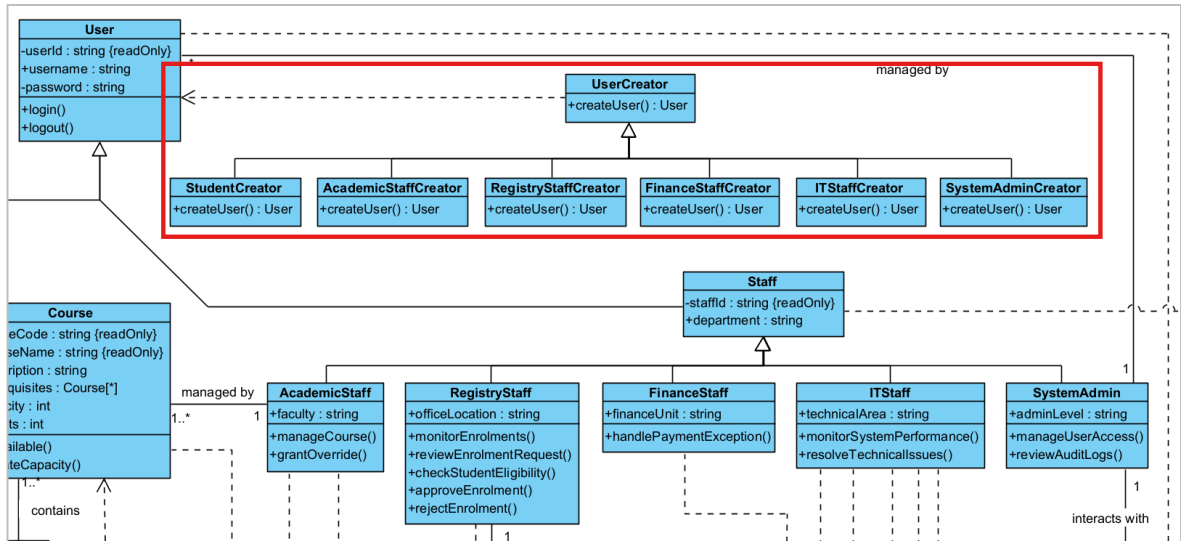


Figure 8: Updated class diagram after applying the Factory Method pattern (new components are boxed in red)

ii. Corresponding C# implementation

The figures below show the corresponding C# implementation of the Factory Method pattern, demonstrating how object creation is delegated to creator classes rather than being instantiated directly within client code. This implementation focuses on demonstrating the structure and behaviour of the Factory Method pattern rather than providing full functional system logic.

- **Product (abstract class):** User acts as the abstract product class that interacts with UserCreator (creator class), defining common attributes and operations shared by all user types. Staff serves as an intermediate abstract class that extends User and provides shared staff-related attributes and behaviours for different staff subclasses.

```
// Abstract base product
public abstract class User
{
    private string userID { get; init; }
    public string username { get; set; }
    private string password { get; set; }
    public void Login() => Console.WriteLine($"[AUTH] User '{username}' successfully logged.");
    public void Logout() => Console.WriteLine($"[AUTH] User '{username}' logged out. Session destroyed.");
}

// Abstract intermediate product
public abstract class Staff : User
{
    private string staffID { get; init; }
    public string department { get; set; }
}
```

Figure 9: Abstract product hierarchy in the Factory Method pattern (User and Staff)

- **Concrete products:** Student, AcademicStaff, RegistryStaff, FinanceStaff, ITStaff, and SystemAdmin act as concrete product classes. These classes inherit from the abstract product structure and represent the specific user and staff types created through the factory method.

```
// Concrete Product 1
public class Student : User
{
    private string studentID { get; init; }
    public string programme { get; set; }
    public StudentType studentType { get; init; }
    private List<Course> selectedCourses = new List<Course>();
    private List<EnrolmentRequest> enrolmentRequests = new List<EnrolmentRequest>();

    public void SelectCourse() { }
    public void SearchCourses() { }
    public void ViewCourseDetails() { }
    public void ViewTimetable() { }
    public void SubmitEnrolment() { }
    public void ViewEnrolmentStatus() { }
}

// Concrete Product 2
public class AcademicStaff : Staff
{
    public string faculty { get; set; }

    public void ManageCourse() { }
    public void GrantOverride() { }
}

// Concrete Product 3
public class RegistryStaff : Staff
{
    public string officeLocation { get; set; }

    public void MonitorEnrolments() { }
    public void ReviewEnrolmentRequest() { }
    public void CheckStudentEligibility() { }
    public void ApproveEnrolment() { }
    public void RejectEnrolment() { }
}

// Concrete Product 4
public class FinanceStaff : Staff
{
    public string financeUnit { get; set; }

    public void HandlePaymentException() { }
}

// Concrete Product 5
public class ITStaff : Staff
{
    public string technicalArea { get; set; }

    public void MonitorSystemPerformance() { }
    public void ResolveTechnicalIssues() { }
}

// Concrete Product 6
public class SystemAdmin : Staff
{
    public string adminLevel { get; set; }

    public void ManageUserAccess() { }
    public void ReviewAuditLogs() { }
}
```

Figure 10: Concrete product classes created through the Factory Method pattern

- **Creator (abstract class):** UserCreator acts as the abstract creator class that declares the factory method responsible for creating User objects. It provides a common interface for object creation while allowing subclasses to determine which concrete product to instantiate.

```
// Abstract Creator
abstract class UserCreator
{
    // The Factory Method
    public abstract User CreateUser();
}
```

Figure 11: Abstract creator class (UserCreator) in the Factory Method pattern

- **Concrete creators:** StudentCreator, AcademicStaffCreator, RegistryStaffCreator, FinanceStaffCreator, ITStaffCreator, and SystemAdminCreator act as concrete creator classes. Each concrete creator overrides the factory method to create and return its corresponding concrete product type, ensuring that object creation is delegated to specialised creator classes rather than being instantiated directly within client code.

```
// Concrete Creators matching each Concrete Product 1-to-1
class StudentCreator : UserCreator
{
    public override User CreateUser() => new Student();
}

class AcademicStaffCreator : UserCreator
{
    public override User CreateUser() => new AcademicStaff();
}

class RegistryStaffCreator : UserCreator
{
    public override User CreateUser() => new RegistryStaff();
}

class FinanceStaffCreator : UserCreator
{
    public override User CreateUser() => new FinanceStaff();
}

class ITStaffCreator : UserCreator
{
    public override User CreateUser() => new ITStaff();
}

class SystemAdminCreator : UserCreator
{
    public override User CreateUser() => new SystemAdmin();
}
```

Figure 12: Concrete creator classes for User and Staff creation

3.2. Singleton pattern

The system requires a consistent and centralised mechanism for recording all system activities, such as login events, enrolment submissions, approvals, and rejections. Without proper control, multiple instances of the logging service could be created, leading to duplicated logs and inconsistent audit records.

The Singleton pattern is applied to the AuditService to ensure that only one instance exists throughout the system. This provides a centralised and consistent logging mechanism for all system operations.

i. Class diagram showing Singleton pattern structure

The figure below shows the implementation of the Singleton pattern in the class diagram. AuditService is updated with a private constructor to prevent external instantiation and a static `_instance` field of type `Lazy<AuditService>` to support thread-safe, lazy initialization of the single shared instance. A public `Instance` property provides global access to this instance, and the `logActivity()` method is used to record system actions. This design improves system quality by ensuring consistent audit logging, reducing duplication, and maintaining reliable traceability across all system operations.

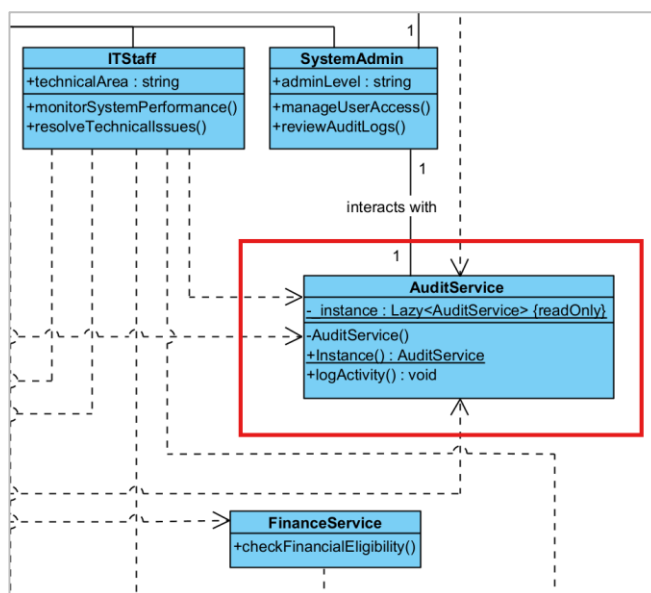


Figure 13: Updated class diagram of AuditService after applying the Singleton pattern (updated AuditService is boxed in red)

ii. Corresponding C# implementation

The figures below show the corresponding C# implementation of the Singleton pattern, demonstrating how a single instance of `AuditService` is created and globally accessed throughout the system. This implementation focuses on demonstrating the structure and behaviour of the Singleton pattern rather than providing full logging functionality.

- **Singleton class:** `AuditService` is implemented as a sealed class to ensure that it cannot be inherited, supporting strict control over its instantiation. The class contains a single responsibility of providing audit logging services across the system.

```
// Singleton class
public sealed class AuditService
{
    // Private constructor prevents external instantiation
}
```

Figure 14: `AuditService` class implementing Singleton Pattern

- **Private constructor:** The constructor of `AuditService` is declared as private to prevent external instantiation using the `new` keyword. This ensures that object creation is fully controlled within the class itself and enforces the Singleton constraint.

```
// Private constructor prevents external instantiation
private AuditService()
{
    Console.WriteLine("[SYSTEM] Audit Service Initialized.");
}
```

Figure 15: Private constructor implementation in Singleton Pattern

- **Static member:** The single instance of `AuditService` is managed using `Lazy<AuditService>`, which ensures thread-safe, lazy instantiation. This guarantees that the instance is only created when it is first accessed, improving performance and ensuring a single shared instance across the system.

```
// Lazy initialization guarantees a single, thread-safe instance
private static readonly Lazy<AuditService> _instance =
    new Lazy<AuditService>(() => new AuditService());
```

Figure 16: Lazy initialization of Singleton Instance (`Lazy<AuditService>`)

- **Static factory method:** A public static property Instance provides controlled global access to the single AuditService instance, allowing all system components to access the same logging service consistently.

```
// Public global access point to the single instance
public static AuditService Instance
{
    get { return _instance.Value; }
}
```

Figure 17: Global access point (instance property) of AuditService

- **Operation:** The LogActivity() method represents the core operation of the AuditService, which is responsible for recording system events such as login actions, enrolment activities, and system changes.

```
// AuditService's operation
public void LogActivity() { }
}
```

Figure 18: Core operation of AuditService

3.3. Adapter pattern

The Course Enrolment System interacts with multiple external legacy systems, including course, student, finance, and HR systems. However, these external systems are not compatible with the internal system.

The Adapter pattern is applied to resolve this incompatibility by allowing different interfaces to work together through conversion into a target interface expected by the system. This ensures that external systems can be reused without modifying their original implementations.

i. Class diagram showing Adapter pattern structure

The figure below shows the changes in the class diagram after applying the Adapter pattern. Multiple adapter classes are introduced, such as CourseInfoAdapter, StudentInfoAdapter, FinanceAdapter, and HRAdapter, each implementing their corresponding target interfaces (ICourseInfoSystem, IStudentInfoSystem, IFinanceSystem, and IHRSystem). These adapters internally delegate requests to their respective legacy systems. This improves system design quality by reducing coupling between the enrolment system and

external systems, increasing flexibility, and enabling seamless integration with legacy components while maintaining a clean separation of concerns.

After applying the Adapter pattern, AcademicStaff and FinanceStaff are shown with dependencies towards LegacyCourseSystem and LegacyFinanceSystem to represent the external systems involved in course management and financial processing. These dependencies illustrate the integration points where external services are required within the system. They are used to provide context of external system involvement rather than representing direct interaction within the Adapter-based design, where communication is instead encapsulated through adapter classes and interfaces.

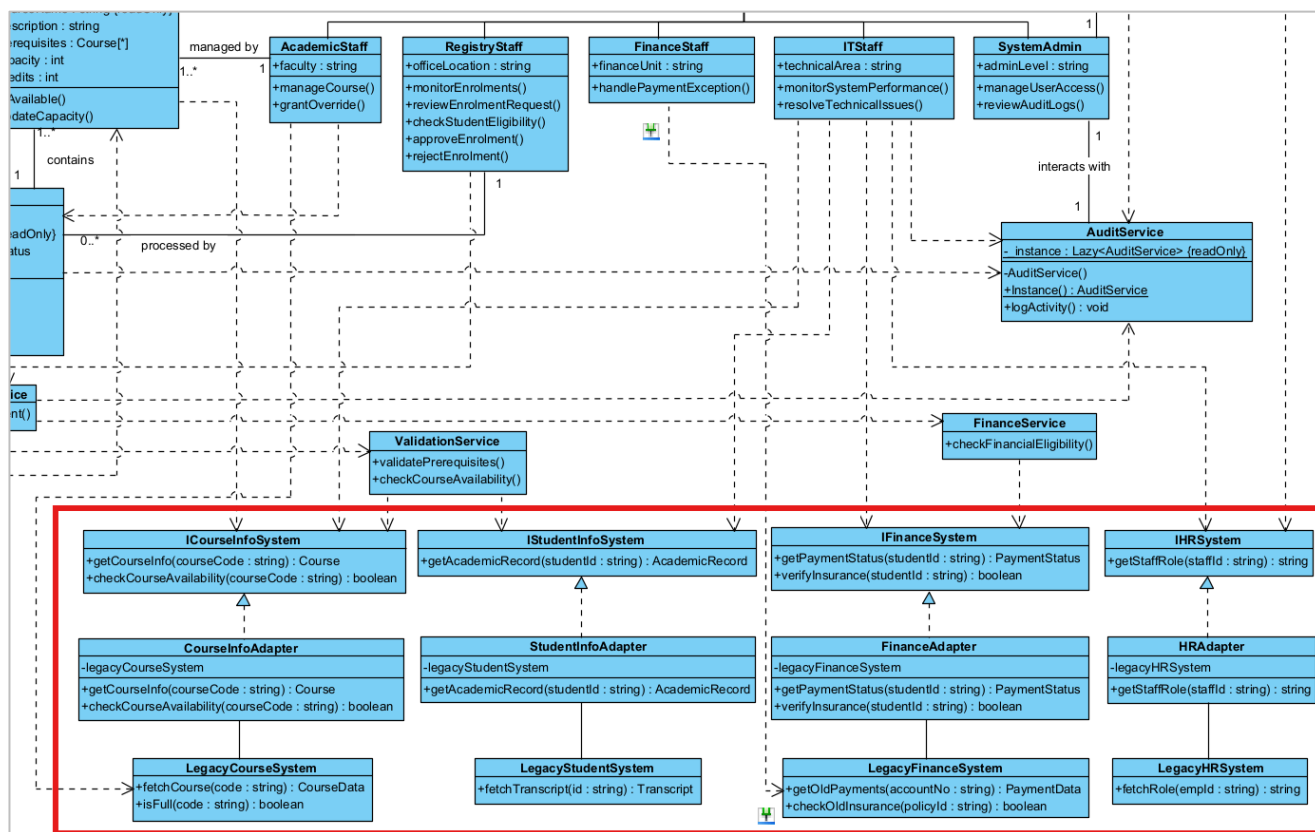


Figure 19: Updated class diagram after applying the Adapter pattern (updated components are boxed in red)

ii. Corresponding C# implementation

The figures below show the corresponding C# implementation of the Adapter pattern, demonstrating how legacy systems are integrated into the Course Enrolment System through adapter classes. This implementation focuses on illustrating how incompatible interfaces are converted into a unified interface expected by the system, rather than providing full system functionality.

- **Target interfaces:** The system defines a set of target interfaces, including `ICourseInfoSystem`, `IStudentInfoSystem`, `IFinanceSystem`, and `IHRSystem`. These interfaces represent the expected operations used by the Course Enrolment System, ensuring a consistent contract for interacting with external services.

```
// Target interface 1
public interface ICourseInfoSystem
{
    Course GetCourseInfo(string courseCode);
    bool CheckCourseAvailability(string courseCode);
}

// Target interface 2
public interface IStudentInfoSystem
{
    AcademicRecord GetAcademicRecord(string studentId);
}

// Target interface 3
public interface IFinanceSystem
{
    PaymentStatus GetPaymentStatus(string studentId);
    bool VerifyInsurance(string studentId);
}

// Target interface 4
public interface IHRSystem
{
    string GetStaffRole(string staffId);
}
```

Figure 20: Target interfaces for Adapter pattern

- **Legacy systems (Adaptees):** The legacy systems, such as `LegacyCourseSystem`, `LegacyStudentSystem`, `LegacyFinanceSystem`, and `LegacyHRSystem`, represent existing external subsystems with incompatible interfaces. These systems cannot be modified and therefore require adaptation to be used within the current system.

```

// Legacy External System 1
public class LegacyCourseSystem
{
    public CourseData FetchCourse(string code) => new CourseData();
    public bool IsFull(string code) => true;
}

// Legacy External System 2
public class LegacyStudentSystem
{
    public Transcript FetchTranscript(string id) => new Transcript();
}

// Legacy External System 3
public class LegacyFinanceSystem
{
    public PaymentData GetOldPayments(string accountNo) => new PaymentData();
    public bool CheckOldInsurance(string policyId) => true;
}

// Legacy External System 3
public class LegacyHRSysytem
{
    public string FetchRole(string empId) => "Lecturer";
}

```

Figure 21: Legacy external systems (adaptees) for Adapter pattern

- **Adapter classes:** Adapter classes such as CourseInfoAdapter, StudentInfoAdapter, FinanceSystemAdapter, and HRSysytemAdapter implement the corresponding target interfaces while internally delegating requests to legacy systems. They act as a bridge between the Course Enrolment System and external subsystems by translating requests into compatible formats and performing necessary data transformation where required. By maintaining references to their respective legacy systems and forwarding requests through the target interfaces, the adapters ensure that the core system interacts only with abstractions, while the complexity of legacy implementations is fully encapsulated.

```

// Adapter 1: Course Info
public class CourseInfoAdapter : ICourseInfoSystem
{
    private LegacyCourseSystem legacyCourseSystem;

    public CourseInfoAdapter() => legacyCourseSystem = new LegacyCourseSystem();

    public Course GetCourseInfo(string courseCode) => legacyCourseSystem.FetchCourse(courseCode);
    public bool CheckCourseAvailability(string courseCode) => !legacyCourseSystem.IsFull(courseCode);
}

// Adapter 2: Student Info
public class StudentInfoAdapter : IStudentInfoSystem
{
    private LegacyStudentSystem legacyStudentSystem;

    public StudentInfoAdapter() => legacyStudentSystem = new LegacyStudentSystem();

    public AcademicRecord GetAcademicRecord(string studentId)
        => legacyStudentSystem.FetchTranscript(studentId);
}

// Adapter 3: Finance
public class FinanceSystemAdapter : IFinanceSystem
{
    private LegacyFinanceSystem legacyFinanceSystem;

    public FinanceSystemAdapter() => legacyFinanceSystem = new LegacyFinanceSystem();

    public PaymentStatus GetPaymentStatus(string studentId)
    {
        string accountNo = ConvertStudentIdToAccountNo(studentId);
        return legacyFinanceSystem.GetOldPayments(accountNo);
    }

    public bool VerifyInsurance(string studentId)
    {
        string policyId = ConvertStudentIdToPolicyId(studentId);
        return legacyFinanceSystem.CheckOldInsurance(policyId);
    }

    private string ConvertStudentIdToAccountNo(string studentId) => "ACC-" + studentId;
    private string ConvertStudentIdToPolicyId(string studentId) => "POL-" + studentId;
}

// Adapter 4: HR
public class HRSYSTEMAdapter : IHRSYSTEM
{
    private LegacyHRSYSTEM legacyHRSYSTEM;

    public HRSYSTEMAdapter() => legacyHRSYSTEM = new LegacyHRSYSTEM();

    public string GetStaffRole(string staffId) => legacyHRSYSTEM.FetchRole(staffId);
}

```

Figure 22: Adapter classes implementing target interfaces in Adapter pattern

These three patterns address key architectural concerns in the system, including object creation, controlled access to shared services, and integration with external legacy systems. The Factory Method pattern improves flexibility and extensibility in user and staff creation. The Singleton pattern ensures a single, consistent instance of the AuditService for system-wide logging. The Adapter pattern enables seamless

Together, these patterns improve the system architecture by reducing coupling, increasing modularity, and enhancing separation of concerns. This results in a more maintainable, scalable, and robust system that follows key object-oriented design principles.



4. Comparative design analysis

4.1. Changes in class structure and responsibility allocation

In the original design, user creation, system logging, and external system communication were tightly coupled within concrete classes. For example, objects such as Student, AcademicStaff, and FinanceStaff were instantiated directly using the new keyword, resulting in duplicated creation logic across the system.

After applying the Factory Method pattern, object creation responsibility was moved to UserCreator and its concrete creator classes such as StudentCreator and AcademicStaffCreator. This centralises creation logic and removes direct dependencies on concrete classes (see [Figure 8](#)).

Similarly, the Singleton pattern refactored AuditService so that system-wide logging responsibility is managed by a single controlled instance, rather than multiple uncontrolled service instances (see [Figure 13](#)).

The Adapter pattern introduced additional structural layers between the system and external services. Instead of direct interaction with legacy systems, responsibilities are now handled by adapter classes such as CourseInfoAdapter and FinanceSystemAdapter (see [Figure 19](#)).

4.2. Improvements in cohesion and reduction in coupling

The original system exhibited high coupling due to direct instantiation and direct dependency on external systems. This made changes difficult because modifications in one class would impact multiple areas of the system.

After applying design patterns, both cohesion and coupling have improved across the system:

- The Factory Method pattern increases cohesion within the user creation module by centralising object creation logic inside UserCreator and its concrete creator classes, while decoupling client code from concrete user implementations.

- The Singleton pattern improves cohesion within the logging subsystem by ensuring that all audit-related responsibilities are handled by a single controlled instance of `AuditService`, preventing scattered and duplicated logging logic across the system. However, although it provides controlled global access to a shared instance, it introduces tight coupling at the architectural level due to hidden global dependencies. Classes that rely on the Singleton instance become implicitly dependent on its concrete implementation, which can make the system harder to test, extend, and refactor.
- The Adapter pattern decouples the system from legacy systems by introducing adapter classes that connect existing target interfaces, such as `ICourseInfoSystem` and `IFinanceSystem`, with incompatible legacy subsystems. Although these interfaces already existed in the initial design, they were not previously linked to external systems. The Adapter pattern bridges this gap by enabling communication between the system and legacy implementations without modifying existing interfaces or legacy code.

As shown in [Figure 23](#), the system demonstrates improved modularity and reduced coupling through the combined application of Factory Method, Singleton, and Adapter patterns, where responsibilities are separated across object creation, global service control, and external system integration layers.

Overall, both cohesion and coupling have improved across the system, but certain design patterns introduce trade-offs. In particular, while the Singleton pattern improves cohesion within the logging subsystem, it also introduces tight coupling at the architectural level due to hidden global dependencies.

4.3. Impact on extensibility and maintainability

The refactored design significantly improves extensibility. In the original system, adding a new user type required modifying multiple classes where object creation logic was tightly coupled to concrete implementations.

With the Factory Method pattern, new user types can be introduced by simply adding a new concrete creator without modifying existing code, aligning with the Open/Closed Principle.

Similarly, the Adapter pattern allows new external systems to be integrated by creating new adapter classes without changing existing system logic, ensuring compatibility with legacy subsystems.

Maintainability is also improved because:

- System responsibilities are clearly separated
- Changes are localised to specific pattern components
- Dependencies are controlled through abstractions layers, reducing system-wide impact of changes

4.4. Consequences for C# implementation complexity

The original implementation was simpler but less scalable, as it relied on direct object creation and direct dependencies on external systems. While this reduced initial coding complexity, it increased maintenance difficulty as the system evolved.

After applying design patterns, the C# implementation became more structured but also slightly more complex due to the introduction of additional abstraction layers, including:

- Factory Method pattern: abstract creator classes (UserCreator) and multiple concrete creator classes for different user types.
- Adapter pattern: dedicated adapter classes for each external legacy system, enabling communication through defined interfaces rather than direct system access.
- Singleton pattern: controlled instance management using Lazy<T> to ensure thread-safe lazy initialization of AuditService.

Although these changes increase initial implementation complexity, they improve long-term maintainability, readability, scalability, and flexibility. This is reflected in the C# codebase, where object creation is delegated to creator classes rather than direct instantiation using `new`, and communication with external systems is managed through adapter classes and interfaces instead of direct dependencies.

4.5. Accommodation of new requirements

The redesigned system supports new requirements more effectively due to its modular and extensible structure:

- New user types (e.g., new staff roles) can be added by introducing a new concrete creator class without modifying existing object creation logic.
- New external systems can be integrated by implementing a new adapter class that conforms to an existing target interface, reducing the impact on the core system.
- New logging requirements can be added within `AuditService` without changing system-wide access patterns due to the Singleton-controlled access point

For example, adding a new `LibraryStaff` user only requires:

- Creating a `LibraryStaff` class
- Creating a `LibraryStaffCreator` class
- No modification to existing factory logic or client code.

Similarly, if a new external subsystem such as a library management system needs to be integrated, a new adapter class can be introduced without affecting existing enrolment system functionality. This demonstrates how the applied design patterns improve the system's ability to accommodate future requirements with minimal impact on existing components.

Overall, the application of design patterns has significantly improved the system design compared to the original implementation. Changes to class structure and responsibility allocation have resulted in clearer separation of concerns, with object creation, logging, and external system integration delegated to specialised components. The introduction

of Factory Method, Singleton, and Adapter patterns has improved cohesion and reduced coupling while increasing extensibility and maintainability through modular and reusable design.

Although the C# implementation became more complex due to additional abstraction layers such as creator classes, adapters, and controlled instance management, this trade-off improves long-term scalability and system quality. Furthermore, the redesigned architecture accommodates new requirements more easily, allowing new user types, external systems, and system functionalities to be introduced with minimal impact on existing code.

5. Critical reflection

5.1. Design trade-offs introduced by design patterns

Although the application of Factory Method, Singleton, and Adapter patterns improved system quality, several design trade-offs were introduced.

The Factory Method pattern improved extensibility and reduced direct dependency on concrete classes, but it increased implementation complexity due to the introduction of multiple creator classes. For example, creating different user types now requires additional creator classes such as StudentCreator, AcademicStaffCreator, and FinanceStaffCreator, which increases the number of classes within the system.

Similarly, the Singleton pattern improved consistency by ensuring that only one instance of AuditService exists across the system, supporting centralised logging and controlled access. However, this pattern introduces tight coupling at the architectural level due to hidden global dependencies. Classes relying on AuditService.Instance become dependent on a specific implementation, making testing, extension, and future replacement more difficult.

The Adapter pattern improved compatibility with external legacy systems without modifying their implementations. However, the introduction of adapter classes increased structural complexity because each external subsystem requires a dedicated adapter and interface mapping. This may result in additional maintenance effort when external systems evolve.

Overall, while these design patterns improved flexibility, modularity, and maintainability, they also introduced additional abstraction layers and implementation complexity, requiring careful balance between design quality and system simplicity.

5.2. Limitations of the final design and implementation

Although the final Course Enrolment System design improves modularity, maintainability, and extensibility through the application of design patterns, several limitations remain.

First, the system continues to depend heavily on external legacy systems for core functions such as course management, finance, student information, and HR processes. While the Adapter pattern improves compatibility, failures, interface changes, or unavailability of external systems may still affect system functionality. For example, AcademicStaff and FinanceStaff rely on external systems to manage courses and handle payment-related exceptions, limiting the system's direct control over these processes.

Second, the use of Factory Method and Adapter patterns increases the number of classes and abstraction layers within the system. Although this improves separation of concerns and extensibility, it also makes the codebase more complex to understand and maintain, particularly when tracing interactions between creators, adapters, interfaces, and concrete implementations.

Additionally, the Singleton implementation of AuditService ensures consistent logging across the system; however, it introduces architectural dependency on a single shared instance. Future requirements involving multiple logging services, distributed systems, or different logging configurations may require further refactoring.

Performance risks may also occur during peak enrolment periods, as the system requires multiple validation processes and interactions with external systems. Despite improved modularity, repeated service calls and dependency on external subsystems may affect response time and overall user experience if optimisation strategies are not implemented.

Finally, as the system expands to support more user roles, business rules, and external services, maintaining consistency between UML models and

implementation may become increasingly challenging, requiring careful design governance to prevent architectural complexity from increasing over time.

5.3. Lessons learned from aligning UML models with C# code

A key lesson learned from this project was that UML class diagrams provide a high-level structural view of the system, but translating them into C# implementation requires careful interpretation of real object-oriented behaviour, dependencies, and language-specific constraints.

During development, mapping the overall system design into C# highlighted the importance of correctly defining class responsibilities and relationships. While the UML model clearly represented entities such as User, Staff, Student, and various staff roles, implementing these in C# required careful decisions regarding inheritance structure, access modifiers, and data encapsulation to ensure that the hierarchy remained consistent and practical in code.

In addition, it was found that UML diagrams do not always include all attributes, operations, or internal structures required for a fully working system. As a result, additional attributes and supporting elements had to be introduced in the C# implementation to ensure correct system behaviour and functional completeness.

Another important learning point was that relationships shown in UML diagrams do not always translate directly into implementation details. Associations between system components needed to be refined when implementing real interactions such as method calls, collections, and dependency flows. This was particularly evident when structuring interactions between service classes, user roles, and external system integrations.

The implementation of design patterns also reinforced the gap between conceptual modelling and real code structure. Although UML diagrams represent patterns such as Factory Method, Singleton, and Adapter in a simplified form, their implementation

in C# required additional considerations such as constructor control, interface enforcement, and dependency management to ensure correct behaviour.

Overall, aligning UML models with C# implementation improved understanding of how system design evolves into working software. It demonstrated that UML serves as a guiding blueprint rather than a fixed specification, and that successful implementation requires continuous refinement to ensure consistency between design intent and actual system behaviour.

6. Conclusion

This report presented the analysis, design, and refinement of the Course Enrolment System for AUT, progressing from Phase 1 requirements modelling to a more structured and maintainable Phase 2 design. The system was improved through refined UML modelling and the application of key GoF design patterns, including Factory Method, Singleton, and Adapter, to address issues related to coupling, extensibility, and integration with external systems.

The application of these design patterns significantly improved the overall software architecture by separating responsibilities across object creation, global service management, and external system integration. As a result, the system achieved higher cohesion, reduced coupling, and improved maintainability and extensibility. Although these improvements introduced additional abstraction layers and increased implementation complexity, they contributed to a more scalable and robust design.

Overall, the final system demonstrates a clear progression from a tightly coupled Phase 1 design to a more modular and flexible architecture in Phase 2. The project also highlights the importance of aligning UML models with real implementation in C#, reinforcing that effective system design requires continuous refinement between conceptual modelling and practical software development.